

✓ Suicide Detection – Final Visualizations and Summary (Quick Run Replication)

This notebook replicates core components from previous modeling notebooks (BERT, RoBERTa, BERTweet, TF-IDF baseline) with 1-epoch fine-tuning on a 10K sample for visualization, comparison, and paper completion.

```
!pip install -q kaggle
from google.colab import files
files.upload()
!mkdir -p ~/.kaggle
!cp kaggle.json ~/.kaggle/
!chmod 600 ~/.kaggle/kaggle.json
!kaggle datasets download -d nikhileswarkomati/suicide-watch
!unzip suicide-watch.zip
import pandas as pd
df = pd.read_csv('Suicide_Detection.csv')
print(f"Dataset shape: {df.shape}")
print(df.head())
```

 Choose Files kaggle.json

- **kaggle.json**(application/json) - 69 bytes, last modified: 4/6/2025 - 100% done

Saving kaggle.json to kaggle.json
 Dataset URL: <https://www.kaggle.com/datasets/nikhileswarkomati/suicide-watch>
 License(s): CC-BY-SA-4.0
 Archive: suicide-watch.zip
 inflating: Suicide_Detection.csv
 Dataset shape: (232074, 3)

Unnamed: 0		text	class
0	2	Ex Wife Threatening SuicideRecently I left my ...	suicide
1	3	Am I weird I don't get affected by compliments...	non-suicide
2	4	Finally 2020 is almost over... So I can never ...	non-suicide
3	8	i need helpjust help me im crying so hard	suicide

```
!pip install -q transformers scikit-learn seaborn
import pandas as pd
import torch
from torch.utils.data import Dataset, DataLoader
from transformers import AutoTokenizer, AutoModelForSequenceClassification, get_linear_schedule_with_warmup
from torch.optim import AdamW
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report, confusion_matrix
import matplotlib.pyplot as plt
import seaborn as sns
from tqdm.auto import tqdm

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print("Using device:", device)

df = pd.read_csv("Suicide_Detection.csv").dropna()
df['label'] = df['class'].map({'non-suicide': 0, 'suicide': 1})
df_small = df.groupby('label', group_keys=False).apply(lambda x: x.sample(n=5000, random_state=42)).reset_index(drop=True)

train_texts, val_texts, train_labels, val_labels = train_test_split(
    df_small['text'].tolist(), df_small['label'].tolist(),
    test_size=0.2, stratify=df_small['label'], random_state=42
)

class RedditDataset(Dataset):
    def __init__(self, encodings, labels):
        self.encodings = encodings
        self.labels = labels

    def __getitem__(self, idx):
        item = {k: torch.tensor(v[idx]) for k, v in self.encodings.items()}
        item['labels'] = torch.tensor(self.labels[idx])
        return item

    def __len__(self):
        return len(self.labels)

def train_and_evaluate(model_name):
```

```

tokenizer = AutoTokenizer.from_pretrained(model_name, normalization=True if "bertweet" in model_name else False)
train_enc = tokenizer(train_texts, truncation=True, padding=True, max_length=128)
val_enc = tokenizer(val_texts, truncation=True, padding=True, max_length=128)
train_ds = RedditDataset(train_enc, train_labels)
val_ds = RedditDataset(val_enc, val_labels)
train_loader = DataLoader(train_ds, batch_size=16, shuffle=True)
val_loader = DataLoader(val_ds, batch_size=16)

model = AutoModelForSequenceClassification.from_pretrained(model_name, num_labels=2).to(device)
optimizer = AdamW(model.parameters(), lr=2e-5)
scheduler = get_linear_schedule_with_warmup(optimizer, num_warmup_steps=0, num_training_steps=len(train_loader))

model.train()
for batch in tqdm(train_loader, desc="Training " + model_name):
    optimizer.zero_grad()
    batch = {k: v.to(device) for k, v in batch.items()}
    outputs = model(**batch)
    loss = outputs.loss
    loss.backward()
    torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)
    optimizer.step()
    scheduler.step()

model.eval()
preds, true = [], []
for batch in val_loader:
    batch = {k: v.to(device) for k, v in batch.items()}
    with torch.no_grad():
        outputs = model(**batch)
    logits = outputs.logits
    preds.extend(torch.argmax(logits, dim=1).cpu().numpy())
    true.extend(batch['labels'].cpu().numpy())

report = classification_report(true, preds, target_names=['Non-suicide', 'Suicide'], output_dict=True)
conf_mat = confusion_matrix(true, preds)
return report, conf_mat

bert_report, bert_conf = train_and_evaluate("bert-base-uncased")
roberta_report, roberta_conf = train_and_evaluate("roberta-base")
bertweet_report, bertweet_conf = train_and_evaluate("vinai/bertweet-base")

def plot_conf_matrix(conf_mat, title):
    sns.heatmap(conf_mat, annot=True, fmt='d', cmap='Blues', xticklabels=['Non-suicide', 'Suicide'], yticklabels=['Non-suicide', 'Suicide'])
    plt.xlabel("Predicted")
    plt.ylabel("Actual")
    plt.title(f"{title} Confusion Matrix")
    plt.show()

def plot_model_comparison(reports, labels):
    precisions = [r['Suicide']['precision'] for r in reports]
    recalls = [r['Suicide']['recall'] for r in reports]
    f1s = [r['Suicide']['f1-score'] for r in reports]

    x = range(len(labels))
    plt.figure(figsize=(10,6))
    plt.bar([i-0.2 for i in x], precisions, width=0.2, label='Precision')
    plt.bar(x, recalls, width=0.2, label='Recall')
    plt.bar([i+0.2 for i in x], f1s, width=0.2, label='F1 Score')
    plt.xticks(x, labels)
    plt.ylabel("Score")
    plt.ylim(0.8, 1.0)
    plt.title("Model Performance (1 Epoch, 10K Sample)")
    plt.legend()
    plt.show()

plot_model_comparison([bert_report, roberta_report, bertweet_report], ["BERT", "RoBERTa", "BERTweet"])
plot_conf_matrix(bert_conf, "BERT")
plot_conf_matrix(roberta_conf, "RoBERTa")
plot_conf_matrix(bertweet_conf, "BERTweet")

```

```

Using device: cuda
<ipython-input-2-4cddd882253c>:19: DeprecationWarning: DataFrameGroupBy.apply operated on the grouping columns. This be
df_small = df.groupby('label', group_keys=False).apply(lambda x: x.sample(n=5000, random_state=42)).reset_index(drop=
tokenizer_config.json: 100% 48.0/48.0 [00:00<00:00, 4.45kB/s]

config.json: 100% 570/570 [00:00<00:00, 72.3kB/s]

vocab.txt: 100% 232k/232k [00:00<00:00, 3.91MB/s]

tokenizer.json: 100% 466k/466k [00:00<00:00, 6.34MB/s]
Xet Storage is enabled for this repo, but the 'hf_xet' package is not installed. Falling back to regular HTTP download.
WARNING:huggingface_hub.file_download:Xet Storage is enabled for this repo, but the 'hf_xet' package is not installed.
model.safetensors: 100% 440M/440M [00:07<00:00, 70.5MB/s]
Some weights of BertForSequenceClassification were not initialized from the model checkpoint at bert-base-uncased and a
You should probably TRAIN this model on a down-stream task to be able to use it for predictions and inference.
Training bert-base-uncased: 100% 500/500 [00:41<00:00, 12.35it/s]

tokenizer_config.json: 100% 25.0/25.0 [00:00<00:00, 3.20kB/s]

config.json: 100% 481/481 [00:00<00:00, 65.2kB/s]

vocab.json: 100% 899k/899k [00:00<00:00, 8.92MB/s]

merges.txt: 100% 456k/456k [00:00<00:00, 6.82MB/s]

tokenizer.json: 100% 1.36M/1.36M [00:00<00:00, 20.3MB/s]
Xet Storage is enabled for this repo, but the 'hf_xet' package is not installed. Falling back to regular HTTP download.
WARNING:huggingface_hub.file_download:Xet Storage is enabled for this repo, but the 'hf_xet' package is not installed.
model.safetensors: 100% 499M/499M [00:04<00:00, 117MB/s]
Some weights of RobertaForSequenceClassification were not initialized from the model checkpoint at roberta-base and are
You should probably TRAIN this model on a down-stream task to be able to use it for predictions and inference.
Training roberta-base: 100% 500/500 [00:41<00:00, 12.10it/s]

config.json: 100% 558/558 [00:00<00:00, 68.6kB/s]

vocab.txt: 100% 843k/843k [00:00<00:00, 8.92MB/s]

bpe.codes: 100% 1.08M/1.08M [00:00<00:00, 32.2MB/s]

tokenizer.json: 100% 2.91M/2.91M [00:02<00:00, 1.13MB/s]
emoji is not installed, thus not converting emoticons or emojis into text. Install emoji: pip3 install emoji==0.6.0
pytorch_model.bin: 100% 543M/543M [00:04<00:00, 128MB/s]
Some weights of RobertaForSequenceClassification were not initialized from the model checkpoint at vinai/bertweet-base
You should probably TRAIN this model on a down-stream task to be able to use it for predictions and inference.
Training vinai/bertweet-base: 100% 500/500 [00:41<00:00, 12.06it/s]

model.safetensors: 100% 543M/543M [00:04<00:00, 118MB/s]

```



